

Math-Routed Sparse Transformer

Architecture Reference

Overview

The Math-Routed Sparse Transformer compiles mathematical expression structure into a sparse attention routing table *before* model inference runs. The central claim is:

$$\boxed{\text{symbolic topology} \longrightarrow \text{bounded neural attention}}$$

Instead of computing full $O(T^2)$ dense attention over T tokens, each token attends only to a fixed- K neighborhood derived from the mathematical structure of the input expression, reducing the attention kernel cost to $O(TK)$.

All three topology modes (symbolic, learned, block) produce the same two tensors that feed the Triton sparse attention kernel:

$$\mathcal{N} \in \mathbb{Z}^{T \times K} \quad (\text{neighbor index table}), \quad V \in \{0, 1\}^{T \times K} \quad (\text{validity mask}).$$

Topology construction is offline. The compiled $(\mathcal{N}, V)$ pair is stored as static buffers and reused across all model forwards without rebuilding.

Notation

Symbol	Meaning
T	number of tokens (math nodes)
d	model dimension
d_h	head dimension, $d_h = d/H$
H	number of attention heads
K	neighbor budget per token
L	number of transformer layers
b	block size (block topology)
B	number of blocks, $B = \lceil T/b \rceil$
R	number of symbolic relation types
d_E	frozen node embedding dimension
d_F	learned edge feature dimension

Token Representations

Each math node i is encoded to a frozen embedding $e_i \in \mathbb{R}^{d_E}$ by the **MathEmbedder**. These embeddings do not change during training. They are used only to build the topology; the model itself operates on learned projections $x_i \in \mathbb{R}^d$.

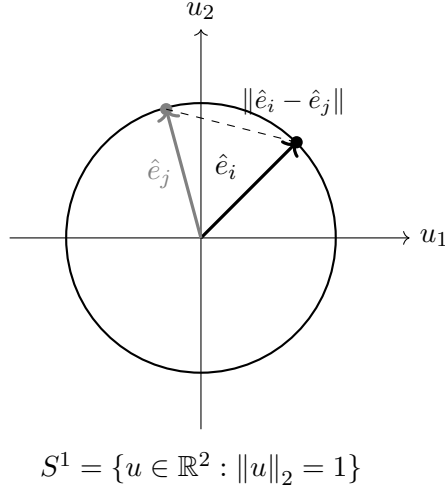
Direction–Magnitude Decomposition

The embedding is decomposed into a unit-norm direction and a scalar magnitude:

$$m_i = \|e_i\|_2, \quad \hat{e}_i = \frac{e_i}{m_i + \varepsilon}, \quad \hat{e}_i \in S^{d_E-1}.$$

The unit sphere $S^{d_E-1} \subset \mathbb{R}^{d_E}$ is $\{u \in \mathbb{R}^{d_E} : \|u\|_2 = 1\}$. Restricting directions to the sphere makes cosine similarity the natural measure of symbolic alignment:

$$\cos \theta_{ij} = \hat{e}_i^\top \hat{e}_j, \quad \|\hat{e}_i - \hat{e}_j\|_2^2 = 2(1 - \cos \theta_{ij}).$$



Projection from Embedding Space to Model Space

The frozen embeddings live in $\mathbb{R}^{d_E}$; the transformer operates in $\mathbb{R}^d$. A learned linear projection maps between the two:

$$x_i^{(0)} = W_{\text{proj}} e_i, \quad W_{\text{proj}} \in \mathbb{R}^{d \times d_E}.$$

For this projection to preserve neighbourhood structure, the $(\hat{e}_i, \hat{e}_j)$ distances before projection should approximately equal those after:

$$(1 - \delta) \|\hat{e}_i - \hat{e}_j\|_2^2 \leq \|R\hat{e}_i - R\hat{e}_j\|_2^2 \leq (1 + \delta) \|\hat{e}_i - \hat{e}_j\|_2^2,$$

which implies:

$$\hat{e}_i^\top \hat{e}_j \approx \hat{z}_i^\top \hat{z}_j, \quad z_i = R\hat{e}_i, \quad \hat{z}_i = \frac{z_i}{\|z_i\|_2 + \varepsilon}.$$

$S^{d_E-1} \xrightarrow{R} S^{d-1} \quad \text{if distances and cosines are preserved within } \delta$

Symbolic Topology

The symbolic topology builder constructs a scored graph over the T tokens by computing R binary relation matrices and combining them with relation-type weights. All relation matrices are computed from the frozen embeddings and the expression structure; none require a model forward pass.

Relation Types

Each relation type r defines a binary matrix $M^{(r)} \in \{0, 1\}^{T \times T}$:

Relation	Definition
M_{ij}^{id}	$\mathbf{1}[i = j]$ (self)
M_{ij}^{dep}	$\mathbf{1}[(i, j) \in \text{AST edges}]$ (symbolic dependency)
M_{ij}^{comp}	$\mathbf{1}[j \text{ is a composition argument of } i]$
M_{ij}^{shp}	$\mathbf{1}[\text{shapes of } i, j \text{ are compatible}]$
M_{ij}^{emb}	$\mathbf{1}[\hat{e}_i^\top \hat{e}_j \geq \tau_{\text{emb}}]$
M_{ij}^{win}	$\mathbf{1}[i - j \leq w]$ (local window)
M_{ij}^{op}	$\mathbf{1}[\text{op}(i) = \text{op}(j)]$ (same operator)
M_{ij}^{mid}	middle-bridge relation (see below)

Weighted Priority Score

Each relation type is assigned a non-negative weight $w_r \geq 0$ and a priority level $p_r \in \mathbb{Z}_{\geq 0}$. The combined priority score is:

$$S_{ij}^{\text{sym}} = \sum_{r=1}^R p_r \cdot M_{ij}^{(r)}.$$

Tokens with higher priority scores are preferred as neighbours. Ties within the same priority level are broken by the weighted score:

$$\tilde{S}_{ij}^{\text{sym}} = \sum_{r=1}^R w_r \cdot M_{ij}^{(r)}.$$

Top- K Neighbour Selection

For each token i , the K highest-scoring column indices are selected:

$$\mathcal{N}_i^{\text{sym}} = \text{TopK}_j(S_{ij}^{\text{sym}}, K), \quad |\mathcal{N}_i^{\text{sym}}| = K.$$

The resulting table $\mathcal{N}^{\text{sym}} \in \mathbb{Z}^{T \times K}$ and validity mask $V^{\text{sym}} \in \{0, 1\}^{T \times K}$ are cached. No $O(T^2)$ mask is materialised at inference time.

Middle-Preserving Topology (v7)

Standard top- K selection tends to concentrate neighbours near the sequence endpoints and the root. Tokens in the middle of a long expression are under-represented — the “lost-in-the-middle” problem.

The middle-bridge relation explicitly connects each token to a window of w tokens centred on the midpoint $T/2$:

$$M_{ij}^{\text{mid}} = \mathbf{1}[|j - \frac{T}{2}| \leq w].$$

This relation is included in the scored top- K build with weight $w_{\text{mid}} = 0.7$ and priority $p_{\text{mid}} = 4$. The combined score becomes:

$$S_{ij} = \alpha \cdot M_{ij}^{\text{id}} + \sum_{r \neq \text{mid}} \beta_r M_{ij}^{(r)} + \gamma \cdot M_{ij}^{\text{mid}}.$$

Because the top- K selection is applied to S_{ij} directly, the resulting topology is fixed- K by construction:

$$|\mathcal{N}_i| = K \quad \forall i, \quad (\text{no runtime truncation needed}).$$

At $T = 1024$, $K = 16$, trees mode, the middle-bridge topology produces:

$$\text{allowed edges} = T \cdot K = 16,384 \quad \text{vs} \quad \text{full} = T^2 = 1,048,576.$$

Learned Topology

The symbolic topology encodes hand-crafted rules. Learned topology replaces the scoring function with a neural network trained to select neighbours that maximise route quality.

Edge Feature Tensor

For each token pair (i, j) , a feature vector is computed from the frozen embeddings and symbolic relation matrices:

$$F_{ij} = [M_{ij}^{\text{id}}, M_{ij}^{\text{dep}}, M_{ij}^{\text{comp}}, M_{ij}^{\text{shp}}, M_{ij}^{\text{mid}}, M_{ij}^{\text{win}}, M_{ij}^{\text{op}}, \hat{e}_i^T \hat{e}_j, \frac{|i-j|}{T}, \frac{i-j}{T}]^T \in \mathbb{R}^{d_F},$$

with $d_F = 10$. The full edge feature tensor is $F \in \mathbb{R}^{T \times T \times d_F}$.

Scorer

A learned scorer maps each edge feature to a scalar score:

$$S_{ij}^{\text{learned}} = f_{\theta}(F_{ij}) \in \mathbb{R}.$$

Top- K neighbour selection proceeds identically to the symbolic case:

$$\mathcal{N}_i^{\text{learned}} = \text{TopK}_j(S_{ij}^{\text{learned}}, K).$$

Scaling Limitation

Building F requires computing all T^2 token pairs. The scoring cost is:

$$\text{cost} = O(T^2 \cdot d_F).$$

For $T = 4096$: $T^2 = 16,777,216$ pairs — this causes an out-of-memory error during topology construction. The block topology (below) resolves this.

Empirical result. On `synthetic_hard/val.jsonl`:

$$Q_{\text{learned}, K=6} = 0.9871 \quad > \quad Q_{\text{hand}, K=16} = 0.9821.$$

The learned scorer achieves higher route accuracy at a smaller neighbour budget than the hand-designed symbolic topology.

Block Topology (v14)

Block topology replaces token-pair scoring with block-pair scoring, reducing the quadratic cost by a factor of b^2 .

Block Partition

The T tokens are divided into B contiguous blocks of size b :

$$B = \lceil T/b \rceil, \quad \text{block } \beta = \{(\beta - 1)b, \dots, \min(\beta b, T) - 1\}.$$

Block Features

Each block β is summarised by a feature vector $\phi_\beta \in \mathbb{R}^{d_B}$ computed from the operator histogram, node-kind histogram, shape summary, and middle-token coverage of the tokens it contains.

Block Scoring

Block pairs are scored with a heuristic function:

$$S_{\beta\gamma}^{\text{block}} = S_{\text{local}}(\beta, \gamma) + S_{\text{bridge}}(\beta, \gamma) + S_{\text{mid}}(\beta, \gamma) - \lambda_d \cdot |\beta - \gamma|.$$

The cost of scoring all block pairs is:

$$O(B^2) = O\left(\frac{T^2}{b^2}\right),$$

a factor of b^2 cheaper than token-pair scoring. For $T = 4096$ and $b = 64$: $B^2 = 4,096$ vs $T^2 = 16,777,216$.

Token Expansion

The top- K_B block neighbours of each block are selected:

$$\mathcal{N}_\beta^{\text{block}} = \text{TopK}_\gamma \left(S_{\beta\gamma}^{\text{block}}, K_B \right).$$

Each token i in block β inherits the token ranges of all selected neighbouring blocks:

$$\mathcal{N}_i^{\text{token}} = \bigcup_{\gamma \in \mathcal{N}_{\beta(i)}^{\text{block}}} \{ t : \gamma b \leq t < \min((\gamma + 1)b, T) \}.$$

This is capped to K tokens per row (matching the fixed- K interface of the other topology modes), with local-window and symbolic-bridge tokens added:

$$\mathcal{N}_i^{\text{final}} = \text{TopK} \left(\mathcal{N}_i^{\text{token}} \cup \mathcal{N}_i^{\text{local}} \cup \mathcal{N}_i^{\text{bridge}}, K \right).$$

$$O(B^2) = O\left(\frac{T^2}{b^2}\right) \longrightarrow \text{no OOM at } T \in \{1024, 2048, 4096\}$$

Sparse Attention Kernel

All three topology modes produce the same interface. The Triton sparse attention kernel consumes the neighbor table directly, without materialising an $O(T^2)$ attention matrix.

Projections

For layer ℓ and head h , the input $X^{(\ell)} \in \mathbb{R}^{T \times d}$ is projected to queries, keys, and values:

$$Q^{(\ell,h)} = X^{(\ell)} W_Q^{(\ell,h)}, \quad K^{(\ell,h)} = X^{(\ell)} W_K^{(\ell,h)}, \quad V^{(\ell,h)} = X^{(\ell)} W_V^{(\ell,h)},$$

$$W_Q^{(\ell,h)}, W_K^{(\ell,h)}, W_V^{(\ell,h)} \in \mathbb{R}^{d \times d_h}.$$

Neighbour-Sparse Attention

For each token i , attention is computed only over its K neighbours. Let $\mathcal{N}_{i,k}$ denote the k -th neighbour index of token i . The raw attention scores are:

$$a_{i,k}^{(\ell,h)} = \frac{q_i^{(\ell,h)\top} k_{\mathcal{N}_{i,k}}^{(\ell,h)}}{\sqrt{d_h}}, \quad k = 1, \dots, K.$$

Valid neighbours are selected by the mask $V_{i,k} \in \{0, 1\}$:

$$p_{i,k}^{(\ell,h)} = \frac{V_{i,k} \exp(a_{i,k}^{(\ell,h)})}{\sum_{k'=1}^K V_{i,k'} \exp(a_{i,k'}^{(\ell,h)})}.$$

The output for token i is the weighted sum over neighbour values:

$$o_i^{(\ell,h)} = \sum_{k=1}^K p_{i,k}^{(\ell,h)} v_{\mathcal{N}_{i,k}}^{(\ell,h)}.$$

Multi-Head Concatenation and Output Projection

The per-head outputs are concatenated and projected:

$$O^{(\ell)} = [o^{(\ell,1)} \dots o^{(\ell,H)}] W_O^{(\ell)}, \quad W_O^{(\ell)} \in \mathbb{R}^{H d_h \times d}.$$

Complexity

Neighbour-sparse attention: $O(T \cdot K \cdot d_h)$ vs Dense attention: $O(T^2 \cdot d_h)$.

At $T = 1024$, $K = 16$, $d_h = 32$: the Triton sparse kernel runs at approximately 0.30 ms versus 1.96 ms for dense attention — a $6.5\times$ kernel-level speedup.

Block-Token Launch Pattern

The default Triton kernel uses a block-token program partition: one Triton program handles $t_{\text{block}} = 2$ consecutive token rows at once (default `block.t=2`). This increases arithmetic intensity per program launch:

$$\text{programs} = \frac{T}{t_{\text{block}}} \cdot H \cdot B_{\text{batch}} \quad \text{vs} \quad T \cdot H \cdot B_{\text{batch}} \quad \text{for the one-token variant.}$$

Transformer Block

Each transformer layer applies pre-norm attention followed by a feed-forward network, with residual connections.

Pre-Norm Attention

$$\tilde{x}_i^{(\ell)} = \text{LayerNorm}(x_i^{(\ell)}),$$

$$x_i^{(\ell+1/2)} = x_i^{(\ell)} + \text{Dropout}\left(o_i^{(\ell)}\right), \quad o_i^{(\ell)} = \sum_h W_O^{(\ell)} [o_i^{(\ell,1)} \dots o_i^{(\ell,H)}]^\top.$$

Feed-Forward Network

$$\text{FFN}(x) = W_2 \text{GELU}(W_1 x + b_1) + b_2, \quad W_1 \in \mathbb{R}^{d_{\text{ff}} \times d}, \quad W_2 \in \mathbb{R}^{d \times d_{\text{ff}}}.$$

$$x_i^{(\ell+1)} = x_i^{(\ell+1/2)} + \text{Dropout}\left(\text{FFN}(\text{LayerNorm}(x_i^{(\ell+1/2)}))\right).$$

Block Output

The full block map for layer ℓ :

$$\mathcal{B}^{(\ell)} : X^{(\ell)}, \mathcal{N}, V \mapsto X^{(\ell+1)}.$$

The composed model with L layers:

$$\mathcal{A}_{\Theta, \mathcal{N}, V} = \mathcal{B}^{(L-1)} \circ \dots \circ \mathcal{B}^{(0)} \circ W_{\text{proj}}.$$

Two-Stage Inference

Topology construction is too expensive to run in every forward pass. At $T = 1024$, symbolic topology build takes 1–2 seconds. The architecture is therefore strictly two-stage:

$$\underbrace{\text{nodes} \rightarrow \mathcal{N}, V}_{\text{Stage 1: offline compile (once)}} \longrightarrow \underbrace{X^{(0)}, \mathcal{N}, V \rightarrow Y}_{\text{Stage 2: inference (many times)}}.$$

After `prepare_static_topology` is called, $\mathcal{N}$ and V are stored as registered CUDA buffers. Every subsequent forward uses `forward_static_fast_path`, which reads directly from the buffers without any topology lookup:

$\text{Stage 2 cost} = O(T \cdot K \cdot d_h \cdot H \cdot L) \quad (\text{no topology construction})$

Training Objectives

Route Classification Loss

The model is trained to predict which expert $y \in \{1, \dots, E\}$ should handle a given math expression. The route logit is computed from the root token:

$$\ell_{\text{route}}(x) = W_{\text{route}} x_0^{(L)} \in \mathbb{R}^E.$$

The training loss is cross-entropy:

$$\mathcal{L}_{\text{route}} = - \sum_{(x, y) \in \mathcal{D}} \log p_{\Theta}(y \mid x), \quad p_{\Theta}(y \mid x) = \text{softmax}(\ell_{\text{route}}(x))_y.$$

Topology Scorer Loss

The learned topology scorer is trained with a combined loss:

$$\mathcal{L}_{\text{scorer}} = \mathcal{L}_{\text{route}} + \lambda_t \mathcal{L}_{\text{teacher}} + \lambda_s \mathcal{L}_{\text{sparsity}},$$

where the teacher agreement term penalises divergence from the dense-attention probability distribution P^{dense} :

$$\mathcal{L}_{\text{teacher}} = \frac{1}{T} \sum_{i=1}^T \text{KL} \left(P_i^{\text{dense}} \parallel P_i^{\text{sparse}, K} \right),$$

and the sparsity term encourages smaller K :

$$\mathcal{L}_{\text{sparsity}} = \frac{1}{T} \sum_{i=1}^T |\mathcal{N}_i|.$$

Route Quality Metric

The primary evaluation metric is route accuracy under sparse attention:

$$Q(K) = \frac{1}{|\mathcal{D}|} \sum_{(x,y) \in \mathcal{D}} \mathbf{1}[\hat{y}_{\text{sparse}, K}(x) = y].$$

Dense agreement measures whether sparse routing produces the same decision as full attention on the same checkpoint weights:

$$Q_{\text{agree}}(K) = \frac{1}{|\mathcal{D}|} \sum_{(x,y) \in \mathcal{D}} \mathbf{1}[\hat{y}_{\text{sparse}, K}(x) = \hat{y}_{\text{dense}}(x)].$$

The v7 checkpoint achieves $Q_{\text{agree}}(16) = 1.0000$ on the current synthetic validation split.

Parameter Update

$$\Theta_{t+1} = \Theta_t - \eta \nabla_{\Theta} \mathcal{L}.$$

Proven Results

Measurement	Value	Config
Dense attention	1.96 ms	$T = 1024$, trees
Triton sparse attention	0.30 ms	$T = 1024$, $K = 16$, trees
Kernel speedup	$6.5\times$	
Dense transformer block	2.398 ms	$T = 1024$, $K = 16$, trees
Cached sparse block	2.017 ms	
Prepared static block	0.941 ms	
Prepared vs dense speedup	$2.55\times$	
Learned $K = 6$ route acc	0.9871	synthetic_hard
Hand $K = 16$ route acc	0.9821	synthetic_hard
v7 $Q(K=16)$	1.0000	synthetic val
v7 $Q_{\text{agree}}(16)$	1.0000	synthetic val
Block topology OOM at $T = 4096$	none	$b = 64$, $B^2 = 4,096$
Token-pair OOM at $T = 4096$	yes	$T^2 = 16,777,216$

The prepared-static block parity threshold has been crossed:

$$p_{\text{blk}} = 0.941 \text{ ms} < d_{\text{blk}} = 2.398 \text{ ms} \quad (T = 1024, K = 16, \text{ trees})$$

The remaining overhead distribution within the prepared block:

$$\frac{p_{\text{attn}}}{p_{\text{blk}}} \approx 34\% \quad (\text{attention kernel}), \quad \frac{p_{\text{non}}}{p_{\text{blk}}} \approx 66\% \quad (\text{LayerNorm, QKV, out_proj, FFN}).$$

The bottleneck has shifted from topology construction to standard transformer block infrastructure.